

OD Estimation on the Sioux Falls Network

Method Documentation and Machine-Learning Pipeline

Implementation: `src/`, `generate_od_dataset.py`, `generate_turning_counts.py`

June 24, 2026

Abstract

This document describes the full pipeline for origin–destination (OD) estimation on the Sioux Falls network: (i) synthetic OD matrix generation with quality metrics, (ii) probabilistic turning-count observation via a route-choice-aware linear assignment operator, and (iii) machine-learning model selection for the inverse problem. The exposition is organized in three parts; a fourth part is reserved for future project stages. Notation is fixed throughout. The primary synthetic generator uses Dirichlet sampling around the base OD distribution, marginal perturbation, and iterative proportional fitting (IPF). Turning counts are generated by K -shortest-path enumeration, multinomial logit route choice, and a fractional assignment matrix $\mathbf{A}_{\text{turn}}$, with optional Poisson observation noise. Implementations live in `src/` and are designed for datasets of up to 10^6 samples using batched, memory-mapped I/O.

Contents

I	Synthetic Data Generation and Evaluation	3
1	Introduction and Problem Setting	4
1.1	The OD matrix	4
1.2	Why synthetic data?	4
1.3	End-to-end pipeline overview	4
1.4	Notation summary	4
2	Generative Methodology: From Base OD to Synthetic Matrices	6
2.1	Structural constraints	6
2.2	Two generations of synthetic generators	6
2.3	Stage 1: Base OD analysis	7
2.4	Stage 2: Base probability distribution	7
2.5	Stage 3: Dirichlet sampling	7
2.6	Stage 4: Demand reconstruction	8
2.7	Stage 5–6: Sparsity and diagonal enforcement	8
2.8	Stage 7: Marginal perturbation	8
2.9	Stage 8: IPF balancing	8
3	Evaluation Metrics and Hyperparameter Selection	9
3.1	Per-matrix metrics	9
3.2	Dataset-level metrics	9
3.3	Hyperparameter grid search	10
3.4	Scalability optimizations	10
4	Implementation Reference (Part I)	11
II	Turning-Count Observation and Dataset Construction	12
5	Network Representation and Turning Movements	13
5.1	Sioux Falls network	13
5.2	Turning movement definition	13
5.3	Two generations of route assignment	13
6	Route Enumeration and Probabilistic Assignment	15
6.1	OD pair indexing	15
6.2	Route catalog: K shortest simple paths	15
6.3	Multinomial logit route choice	16

7	Fractional Assignment Matrix and Forward Model	17
7.1	Route-level turn incidence	17
7.2	Fractional turning assignment matrix	17
7.3	Forward model (clean turning counts)	17
7.4	Identifiability and rank analysis	18
8	Observation Noise and Dataset Assembly	19
8.1	Observation noise models	19
8.2	End-to-end Phase 2 pipeline	20
8.3	Scalability: why the system restarted and how it was fixed	20
8.4	Outputs and deliverables	20
8.5	Validation	20
8.6	Sensitivity analysis	21
8.7	Part II evaluation highlights	21
9	Implementation Reference (Part II)	22
III	Machine Learning Model Selection and Training	23
10	Inverse Problem Formulation	24
10.1	Problem statement	24
10.2	Learning paradigms considered	24
11	Candidate Models and Selection Rationale	25
11.1	Model zoo	25
11.2	Why Ridge + Elastic Net as primary baselines	25
11.3	Why MLP as nonlinear extension	26
11.4	Why not simulation-based or equilibrium models for training labels?	26
12	Training Protocol and Evaluation	27
12.1	Recommended data split	27
12.2	Feature and target preprocessing	27
12.3	Loss functions	27
12.4	ML evaluation metrics	27
12.5	Hyperparameter tuning (Part III)	28
12.6	Planned implementation (Part III)	28
IV	Future Work and Extensions	29
13	Roadmap for Subsequent Chapters	30
	Conclusion	31

Part I

Synthetic Data Generation and Evaluation

Chapter 1

Introduction and Problem Setting

1.1 The OD matrix

Let $N = 24$ be the number of traffic analysis zones in Sioux Falls. The *base* OD matrix

$$\mathbf{T}^{\text{base}} = (T_{ij}^{\text{base}})_{i,j=1}^N \in \mathbb{R}_{\geq 0}^{N \times N} \quad (1.1)$$

records daily trips from origin zone i to destination zone j . We observe a single matrix `EstimatedODMatrix.npy`; the goal is to train models that recover OD demand from turning-count observations.

Marginals and grand total.

$$p_i^{\text{base}} = \sum_{j=1}^N T_{ij}^{\text{base}} \quad (\text{productions}), \quad (1.2)$$

$$a_j^{\text{base}} = \sum_{i=1}^N T_{ij}^{\text{base}} \quad (\text{attractions}), \quad (1.3)$$

$$G = \sum_{i,j} T_{ij}^{\text{base}} = \sum_i p_i^{\text{base}} = \sum_j a_j^{\text{base}}. \quad (1.4)$$

1.2 Why synthetic data?

Machine learning requires many labeled examples. We generate K synthetic matrices $\{\mathbf{T}^{\text{syn},(k)}\}_{k=1}^K$ that:

- preserve total demand G and sparsity structure;
- retain dominant travel patterns but explore realistic variation;
- pair with turning counts via a known forward model for supervised training.

1.3 End-to-end pipeline overview

1.4 Notation summary

Part I: $\mathbf{T}^{\text{base}} \rightarrow \text{Dirichlet+IPF} \rightarrow \{\mathbf{T}^{\text{syn},(k)}\}_{k=1}^K$ (shape $K \times 24 \times 24$)
Part II: $\mathbf{T}^{\text{syn},(k)} \rightarrow \mathbf{y}^{(k)} \rightarrow \mathbf{x}^{(k)} = \mathbf{A}_{\text{turn}}\mathbf{y}^{(k)} + \boldsymbol{\epsilon}^{(k)}$ (K -paths + logit + fractional $\mathbf{A}_{\text{turn}}$)
Part III: train $f : \mathbf{x} \mapsto \mathbf{y}$ on $(\mathbf{X}, \mathbf{Y})$; evaluate on held-out synthetics

Figure 1.1: Three-part pipeline. Part IV (future) will cover deployment, real-data validation, and equilibrium extensions.

Table 1.1: Primary notation used throughout this document.

Symbol	Meaning
$\mathbf{T}^{\text{base}}$	Observed base OD matrix
$\mathbf{T}^{\text{syn}}, \mathbf{T}^{\text{syn},(k)}$	Synthetic OD (single / k -th replicate)
$\mathbf{p}^{\text{base}}, \mathbf{a}^{\text{base}}$	Base productions and attractions
$\mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}}$	IPF target marginals (sum to G)
$p_i^{\text{syn}}, a_j^{\text{syn}}$	Achieved marginals of $\mathbf{T}^{\text{syn}}$
$\mathbf{p}^{\text{prob}}$	Base OD probability vector ($ \mathcal{S} = 552$ nonzero of 576)
$\mathbf{S}, \mathbf{X}^{(t)}$	IPF seed matrix / iterate at step t
$\mathbf{M}$	Binary sparsity mask from $\mathbf{T}^{\text{base}}$
$\mathbf{y}, \mathbf{Y}$	Flattened OD vector / batch ($N^2 = 576$)
$\mathbf{x}, \mathbf{X}$	Turning-count vector / batch ($M = 178$); clean or noisy
$\mathbf{x}^{\text{clean}}, \mathbf{x}^{\text{obs}}$	Clean turning counts / noisy observations
$\mathbf{A}_{\text{turn}}$	Fractional turning assignment matrix (178×576)
$\mathbf{A}_{\text{turn}}^{\text{bin}}$	Binary (shortest-path) assignment matrix
$\mathcal{R}_{ij}$	Set of enumerated routes for OD pair (i, j)
K	Number of shortest simple paths per OD pair
θ	Logit sensitivity (cost scale in route choice)
c_r, U_r, P_r	Route cost, utility, choice probability
$\mathbf{b}_r$	Binary turn-incidence vector for route r
α, δ	Dirichlet concentration / marginal perturbation level

Chapter 2

Generative Methodology: From Base OD to Synthetic Matrices

2.1 Structural constraints

Sparsity support.

$$\mathcal{S} = \{(i, j) : T_{ij}^{\text{base}} > 0\}, \quad M_{ij} = \mathbb{K}[(i, j) \in \mathcal{S}]. \quad (2.1)$$

All generators enforce $T_{ij}^{\text{syn}} = 0$ for $(i, j) \notin \mathcal{S}$.

Zero intrazonal trips. $T_{ii}^{\text{syn}} = 0$ for all i .

2.2 Two generations of synthetic generators

The project implements two generators. Table 2.1 compares them; the **Dirichlet+IPF** pipeline (`src/generate_od_dataset.py`) is the recommended default.

Table 2.1: Comparison of synthetic OD generation methods.

Criterion	Legacy: uniform seed noise (<code>synthetic_od.py</code>)	Current: Dirichlet + IPF (<code>generate_od_dataset.py</code>)
Seed construction	$T_{ij}^{\text{base}} \cdot \eta_{ij}, \eta_{ij} \sim \mathcal{U}(0.8, 1.2)$	$\tilde{\mathbf{p}} \sim \text{Dir}(\alpha \mathbf{p}^{\text{prob}}), \tilde{\sim} = G\tilde{\mathbf{p}}$
Marginal targets	Perturb $\mathbf{p}^{\text{base}}, \mathbf{a}^{\text{base}}$; rescale to G	Same marginal perturbation (configurable δ)
Balancing	IPF to match targets	IPF with Dirichlet sample as seed
Typical ρ vs. base	≈ 0.97 (high, limited diversity)	≈ 0.74 at $\alpha = 500$ (richer ML training)
Pairwise diversity	$\bar{d} \approx 139$	$\bar{d} \approx 545$
Scalability	Multiprocessing; 100k in ~ 5 s	Multiprocessing; 50k in ~ 2.4 s; 1M in ~ 1 min
Hyperparameter search	Manual	Grid over α, δ built-in

Why Dirichlet+IPF was selected. The legacy generator produces synthetics that are almost rescaled copies of $\mathbf{T}^{\text{base}}$ ($\rho \approx 0.97$). For ML, labels must span enough of the 576-dimensional OD space without violating transport realism. Dirichlet sampling on the base probability simplex provides a *principled* way to draw alternative trip tables that remain close

to $\mathbf{T}^{\text{base}}$ when α is large and become more diverse when α is small. IPF then enforces zone-level consistency via perturbed marginals. This separates *distributional* diversity (Dirichlet) from *aggregate* realism (marginals + sparsity).

2.3 Stage 1: Base OD analysis

Given $\mathbf{T}^{\text{base}}$, compute:

- grand total G ;
- production vector $\mathbf{p}^{\text{base}}$ and attraction vector $\mathbf{a}^{\text{base}}$;
- sparsity fraction ($\approx 4.2\%$ zeros for Sioux Falls);
- top 10 / top 20 OD pairs by flow.

Implementation: `src/od_analysis.py`. Outputs `base_od_analysis.json` and CSV summaries.

2.4 Stage 2: Base probability distribution

Flatten $\mathbf{T}^{\text{base}}$ (row-major, C-order) to a vector of length $N^2 = 576$ and normalize:

$$p_k = \frac{T_k^{\text{base}}}{G}, \quad \sum_{k=1}^{576} p_k = 1, \quad (2.2)$$

with $p_k = 0$ for off-support cells. Only the $|\mathcal{S}| = 552$ positive entries participate in sampling. Implementation: `src/od_probability.py`.

2.5 Stage 3: Dirichlet sampling

Draw a probability vector on the support:

$$\tilde{\mathbf{p}} \sim \text{Dirichlet}(\alpha \mathbf{p}_{\mathcal{S}}^{\text{prob}}), \quad (2.3)$$

where $\mathbf{p}_{\mathcal{S}}^{\text{prob}}$ is the base probability restricted to $\mathcal{S}$, and $\alpha > 0$ is a *concentration* parameter.

Effect of α .

- **Small** α (e.g. 100): higher variance, lower correlation with $\mathbf{T}^{\text{base}}$, greater ML diversity.
- **Large** α (e.g. 2000): samples concentrate near $\mathbf{p}^{\text{prob}}$, similar to legacy generator.
- **Default** $\alpha = 500$: balance between realism ($\rho \approx 0.74$) and diversity ($\bar{d} \approx 545$).

Table 2.2: Dirichlet concentration α : trade-offs observed on Sioux Falls ($K = 5000$, $\delta = 0.20$).

α	Mean ρ vs. base	Mean pairwise distance	Recommendation
100	lower (~ 0.65)	highest	exploratory / high-diversity ML
250	moderate	high	alternative if $\alpha = 500$ too conservative
500	≈ 0.74	≈ 545	default production setting
1000	higher	moderate	closer to base; less label noise
2000	highest ($\sim 0.90+$)	lowest	ablation / sanity check only

2.6 Stage 4: Demand reconstruction

Rescale sampled probabilities to trip counts and reshape to $N \times N$:

$$\tilde{T}_{ij} = G \cdot \tilde{p}_{\ell(i,j)}. \quad (2.4)$$

2.7 Stage 5–6: Sparsity and diagonal enforcement

Apply mask and zero diagonal *before* IPF (seed preparation):

$$S_{ij} \leftarrow \tilde{T}_{ij} \cdot M_{ij}, \quad S_{ii} \leftarrow 0, \quad S_{ij} \leftarrow S_{ij} + \varepsilon M_{ij}, \quad \varepsilon = 10^{-6}. \quad (2.5)$$

The floor ε prevents division-by-zero in IPF scaling on the support.

2.8 Stage 7: Marginal perturbation

Independent multiplicative shocks on zone totals, then rescaling to G :

$$\tilde{p}_i = p_i^{\text{base}}(1 + \xi_i^p), \quad \xi_i^p \sim \mathcal{U}(-\delta, \delta), \quad (2.6)$$

$$\tilde{a}_j = a_j^{\text{base}}(1 + \xi_j^a), \quad \xi_j^a \sim \mathcal{U}(-\delta, \delta), \quad (2.7)$$

$$p_i^{\text{tgt}} = G \tilde{p}_i / \sum_{i'} \tilde{p}_{i'}, \quad a_j^{\text{tgt}} = G \tilde{a}_j / \sum_{j'} \tilde{a}_{j'}. \quad (2.8)$$

Default $\delta = 0.20$; grid search tests $\delta \in \{0.05, 0.10, 0.20, 0.30\}$.

Key distinction. $\mathbf{p}^{\text{tgt}}$ and $\mathbf{a}^{\text{tgt}}$ are *IPF targets*, not the final row/column sums of $\mathbf{T}^{\text{syn}}$. IPF induces cell-level flows from the Dirichlet seed subject to these targets.

2.9 Stage 8: IPF balancing

Given seed $\mathbf{S}$ and targets, IPF alternates row and column scaling (Furness/Deming–Stephan). Let $\mathbf{X}^{(t)}$ denote the iterate after t cycles:

$$X_{ij}^{(t+\frac{1}{2})} = X_{ij}^{(t)} \cdot \frac{p_i^{\text{tgt}}}{\sum_{j'} X_{ij'}^{(t)}}, \quad (2.9)$$

$$X_{ij}^{(t+1)} = X_{ij}^{(t+\frac{1}{2})} \cdot \frac{a_j^{\text{tgt}}}{\sum_{i'} X_{i'j}^{(t+\frac{1}{2})}}. \quad (2.10)$$

Stop when marginal error $e^{(t)} < \tau = 10^{-3}$ or after $t_{\text{max}} = 1000$ iterations. Post-process: $\mathbf{T}^{\text{syn}} = \mathbf{X}^{(t)} \odot \mathbf{M}$ with zero diagonal.

Algorithm 1: Dirichlet+IPF synthetic OD (one replicate k)

1. Build $\mathbf{p}^{\text{prob}}$ from $\mathbf{T}^{\text{base}}$; cache mask $\mathbf{M}$
2. Draw $\tilde{\mathbf{p}} \sim \text{Dir}(\alpha \mathbf{p}_S^{\text{prob}})$; form seed $\mathbf{S} = G \text{reshape}(\tilde{\mathbf{p}})$
3. Apply sparsity, diagonal zero, floor ε
4. Draw perturbed targets $(\mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}})$ with level δ ; rescale to G
5. $\mathbf{T}^{\text{syn},(k)} \leftarrow \text{IPF}(\mathbf{S}, \mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}})$
6. Project onto support; zero diagonal

Figure 2.1: Production synthetic OD generator (`src/od_generator.py`).

Chapter 3

Evaluation Metrics and Hyperparameter Selection

3.1 Per-matrix metrics

For each $\mathbf{T}^{\text{syn}}$, with $d_{ij} = T_{ij}^{\text{syn}} - T_{ij}^{\text{base}}$:

Similarity.

$$\text{MAE} = \frac{1}{N^2} \sum_{i,j} |d_{ij}|, \quad \text{RMSE} = \sqrt{\frac{1}{N^2} \sum_{i,j} d_{ij}^2}, \quad (3.1)$$

$$\text{RelErr} = \frac{\sum_{i,j} |d_{ij}|}{G}, \quad \rho = \text{corr}(\text{vec}(\mathbf{T}^{\text{base}}), \text{vec}(\mathbf{T}^{\text{syn}})). \quad (3.2)$$

Marginal drift from base.

$$E_p = \frac{1}{N} \sum_i \left| \frac{p_i^{\text{syn}} - p_i^{\text{base}}}{p_i^{\text{base}} + 10^{-9}} \right|, \quad E_a = \frac{1}{N} \sum_j \left| \frac{a_j^{\text{syn}} - a_j^{\text{base}}}{a_j^{\text{base}} + 10^{-9}} \right|. \quad (3.3)$$

Structural validity. New connections $N_{\text{new}} = 0$, diagonal violation $D_{\text{diag}} = 0$, sparsity matches $\mathbf{T}^{\text{base}}$.

IPF diagnostics. Iteration count, final error e^* , convergence flag.

3.2 Dataset-level metrics

For batch $\{\mathbf{T}^{\text{syn},(k)}\}_{k=1}^K$:

- **Pairwise diversity:** sampled mean/min/max L1 distance between synthetics;
- **Cell variability:** per-OD mean, std, coefficient of variation;
- **Demand consistency:** $\text{CV}_G = \text{std}(\sum_{ij} T_{ij}^{(k)}) / \bar{G} \approx 0$;
- **PCA:** explained variance; components needed for 90%;
- **KMeans:** inertia for $k \in \{2, 5, 10\}$ to detect multimodal structure.

For $K > 10^5$, metrics are computed on a random subset (default 10,000) for tractability.

3.3 Hyperparameter grid search

`src/hyperparameter_search.py` evaluates all pairs (α, δ) on a smaller batch (default 200 samples per config) and ranks configurations by a composite score:

$$\text{score} = \frac{\bar{d}}{200} + \overline{\text{CV}}_{\text{cell}} + \frac{n_{90\%}}{20} - 2 \max(0, \bar{\rho} - 0.85), \quad (3.4)$$

rewarding diversity and PCA spread while penalizing excessive correlation with $\mathbf{T}^{\text{base}}$. Recommended defaults from production runs: $\alpha = 500$, $\delta = 0.20$.

3.4 Scalability optimizations

For $K = 10^6$ samples:

- multiprocessing (`--workers`, default all CPU cores);
- `float32` storage (~ 2.3 GB vs. 4.6 GB);
- per-index RNG seed + k for reproducibility independent of worker count;
- in-place IPF and masking; no IPF history stored at scale;
- sampled evaluation (`--eval-samples 10000`); skip plots (`--skip-plots`).

Benchmark: 50,000 matrices in ≈ 2.4 s on 20 cores; 1,000,000 estimated at ≈ 45 –60 s generation.

Chapter 4

Implementation Reference (Part I)

Table 4.1: Part I: key modules and outputs.

Module / script	Role
<code>generate_od_dataset.py</code>	CLI orchestrator
<code>src/od_analysis.py</code>	Base OD statistics
<code>src/od_probability.py</code>	Probability vector on support
<code>src/dirichlet_sampler.py</code>	Dirichlet draw + demand reconstruction
<code>src/marginal_perturbation.py</code>	Target marginals
<code>src/ipf.py</code>	IPF balancing
<code>src/od_generator.py</code>	Parallel batch generation
<code>src/od_metrics.py</code>	Per-matrix and fast aggregate metrics
<code>src/dataset_evaluation.py</code>	PCA, KMeans, diversity
<code>src/hyperparameter_search.py</code>	Grid search over (α, δ)
<code>synthetic_od.npy</code>	Output array shape $(K, 24, 24)$
<code>dataset_metrics.json</code>	Evaluation summary
<code>dataset_summary.md</code>	Human-readable report

Example command (1M samples).

```
python generate_od_dataset.py \  
  --input EstimatedODMatrix.npy \  
  --samples 1000000 --alpha 500 --perturbation 0.20 \  
  --skip-plots --skip-base-analysis
```

Part II

Turning-Count Observation and Dataset Construction

Chapter 5

Network Representation and Turning Movements

5.1 Sioux Falls network

The road network is parsed from `sioux-falls.net.xml` (SUMO format) into a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ using NetworkX. **No traffic simulation** (SUMO, MATSim, etc.) is used at runtime—only static network topology and connection rules.

- $|\mathcal{V}| = 24$ junctions $J_1, \dots, J_{24}$, mapped one-to-one to zones;
- $|\mathcal{E}| = 76$ directed regular edges with length L_e and free-flow speed;
- 384 SUMO `<connection>` elements defining legal edge-to-edge turns.

Implementation: `src/network_parser.py`.

5.2 Turning movement definition

A turning movement is a consecutive edge pair $(e^{\text{in}}, e^{\text{out}})$ at junction v :

$$\omega(e^{\text{in}}) = v = \alpha(e^{\text{out}}), \quad e^{\text{in}} \neq e^{\text{out}}, \quad (5.1)$$

where $\omega(\cdot)$ and $\alpha(\cdot)$ denote the head and tail junction of a directed edge. Movement m is *admissible* if $(e^{\text{in}}, e^{\text{out}})$ is both a valid junction turn and an allowed SUMO connection. This yields $M = 178$ movements (not 384, because only physically consistent, non-U-turn connections are retained). Each movement receives a unique index $m \in \{0, \dots, M - 1\}$ stored in `turning_id_map`. Implementation: `src/turning_movements.py`; catalog exported as `turning_movements.csv`.

5.3 Two generations of route assignment

The project implements two forward models from OD to turning counts. Table 5.1 compares them; the ***K*-shortest-path + logit** pipeline (`generate_turning_counts.py`) is the recommended default for machine-learning dataset generation.

Why Phase 2 was selected. Phase 1 assigns every OD pair to a single shortest path. Real drivers split across feasible alternatives; a binary $\mathbf{A}_{\text{turn}}$ therefore (i) understates route diversity, (ii) lowers the rank and information content of the forward operator, and (iii) produces turning-count batches with artificially high pairwise correlation. Phase 2 retains the *linear* forward model $\mathbf{x} = \mathbf{A}_{\text{turn}}\mathbf{y}$ —essential for scalable supervised learning—while replacing deterministic

Table 5.1: Comparison of route assignment and turning-count generation methods.

Criterion	Phase 1: binary shortest path (<code>generate_dataset.py</code>)	Phase 2: probabilistic (current) (<code>generate_turning_counts.py</code>)
Route set	Single shortest path π_{ij} per OD	Up to K simple shortest paths $\mathcal{R}_{ij}$
Route choice	Deterministic (all-or-nothing)	Multinomial logit on path costs
$\mathbf{A}_{\text{turn}}$ entries	Binary: $A_{m,\ell} \in \{0, 1\}$	Fractional: $A_{m,\ell} \in [0, 1]$
$\text{rank}(\mathbf{A}_{\text{turn}})$	122 (nullity 454)	174 (nullity 402)
Route diversity	None (same path always)	Controlled by (K, θ)
Observation noise	None	None / Gaussian / Poisson
Scalability at $K_{\text{samples}} = 10^6$	Full in-memory load	Batched memory-mapped I/O
Typical use	Baseline / ablation	Production ML labels

routing with a classical random utility model (multinomial logit) over a finite route set. The result is a fractional $\mathbf{A}_{\text{turn}}$ that is still matrix–vector multiplyable at 10^6 samples but encodes richer route-choice structure.

Chapter 6

Route Enumeration and Probabilistic Assignment

6.1 OD pair indexing

All $N^2 = 576$ zone pairs are indexed in row-major (C) order:

$$\ell(i, j) = (i - 1)N + (j - 1), \quad y_{\ell(i, j)} = T_{ij}, \quad i, j \in \{1, \dots, N\}. \quad (6.1)$$

The flattened OD vector is $\mathbf{y} = (y_0, \dots, y_{575})^\top \in \mathbb{R}_{\geq 0}^{576}$. Implementation: `src/od_pairs.py`.

6.2 Route catalog: K shortest simple paths

For each ordered zone pair (i, j) with $i \neq j$, let s_i and t_j denote the corresponding junction nodes in $\mathcal{G}$. Define the *route catalog*

$$\mathcal{R}_{ij} = \{r_1, \dots, r_{K_{ij}}\}, \quad K_{ij} \leq K, \quad (6.2)$$

where each route r is an edge sequence $\pi = (e_1, \dots, e_L)$ obtained from `networkx.shortest_simple_paths`, which yields simple paths in non-decreasing cost order. Edge cost is either length L_e (default) or free-flow travel time $T_e = L_e/v_e$.

The cost of route r is

$$c_r = \sum_{e \in r} w_e, \quad w_e \in \{L_e, T_e\}. \quad (6.3)$$

Default $K = 5$; sensitivity analysis tests $K \in \{1, 3, 5, 10\}$. Implementation: `src/k_shortest_paths.py`; full catalog saved as `route_catalog.pkl`.

Table 6.1: Route enumeration methods: pros and cons.

Method	Pros	Cons
Single shortest path	Fast; deterministic baseline	Zero route diversity
K shortest <i>simple</i> paths (chosen)	Captures discrete alternatives; no loops; cost-ordered	K hyperparameter; enumeration cost grows with network
K shortest <i>with</i> loops	More paths	Unrealistic cycling; excluded
Random walk sampling	Cheap	No cost ordering; hard to interpret
Simulation-based routing	Realistic congestion	Breaks linear model; slow at 10^6 labels

Why K shortest simple paths. Yen’s / NetworkX simple-path enumeration provides a *simulation-free*, reproducible set of cost-ranked alternatives. Unlike user equilibrium, it preserves a fixed route catalog per OD pair, so $\mathbf{A}_{\text{turn}}$ remains constant across the dataset and the forward map stays linear in $\mathbf{y}$.

6.3 Multinomial logit route choice

Given $\mathcal{R}_{ij}$ with costs $\{c_r\}$, assign utility and probability

$$U_r = -\theta c_r, \quad (6.4)$$

$$P_r = \frac{\exp(U_r)}{\sum_{r' \in \mathcal{R}_{ij}} \exp(U_{r'})} = \frac{\exp(-\theta c_r)}{\sum_{r' \in \mathcal{R}_{ij}} \exp(-\theta c_{r'})}, \quad (6.5)$$

where $\theta > 0$ is a *sensitivity* parameter. Probabilities are computed via a numerically stable softmax (subtract $\max_r U_r$ before exponentiation). Default $\theta = 0.1$; sensitivity grid $\theta \in \{0.05, 0.1, 0.2, 0.5\}$.

Interpretation of θ .

- **Small θ** (e.g. 0.05): utilities are flat; routes split more evenly $\Rightarrow$ higher route diversity and higher rank($\mathbf{A}_{\text{turn}}$).
- **Large θ** (e.g. 0.5): probability mass concentrates on minimum-cost routes $\Rightarrow$ Phase 2 collapses toward Phase 1.

Implementation: `src/route_choice.py`.

Table 6.2: Route choice models: pros and cons.

Model	Pros	Cons
All-or-nothing (Phase 1)	Trivial; rank-122 baseline	Unrealistic; low ML signal
Multinomial logit (chosen)	Standard random utility; closed form; one parameter	IIA assumption; no congestion feedback
Nested logit	Captures route correlation	More parameters; heavier calibration
Mixed logit	Random taste heterogeneity	Simulation required for choice probs
User equilibrium	Network-consistent flows	Nonlinear; not fixed $\mathbf{A}_{\text{turn}}$
Proportional to $1/c_r$	Simple	No random utility foundation

Why multinomial logit. Logit is the canonical closed-form discrete choice model (Ortúzar and Willumsen). With a single scale parameter θ it interpolates between uniform route splitting ($\theta \rightarrow 0$) and deterministic shortest-path assignment ($\theta \rightarrow \infty$), enabling controlled sensitivity analysis without re-running a traffic simulator.

Chapter 7

Fractional Assignment Matrix and Forward Model

7.1 Route-level turn incidence

For route $r = (e_1, \dots, e_L)$, define consecutive turns $\Theta_r = \{(e_k, e_{k+1})\}_{k=1}^{L-1}$. The binary turn-incidence vector $\mathbf{b}_r \in \{0, 1\}^M$ has

$$b_{r,m} = \mathbb{1}[m \text{ corresponds to some turn in } \Theta_r]. \quad (7.1)$$

Implementation: `src/fractional_assignment.py`.

7.2 Fractional turning assignment matrix

Let $\mathbf{A}_{\text{turn}} \in [0, 1]^{M \times N^2}$ denote the *probabilistic* assignment matrix. For OD index $\ell = \ell(i, j)$:

$$A_{\text{turn},m,\ell} = \sum_{r \in \mathcal{R}_{ij}} P_r b_{r,m}. \quad (7.2)$$

Column ℓ sums to the total probability mass on routes for pair (i, j) (unity when routes exist). Entries lie in $[0, 1]$; shape (178, 576) with 7,326 nonzero entries (vs. 1,324 in Phase 1). Implementation: `src/fractional_assignment.py`; saved as `A_turn.npy`.

Relation to Phase 1. The binary matrix $\mathbf{A}_{\text{turn}}^{\text{bin}}$ satisfies

$$A_{m,\ell}^{\text{bin}} = \mathbb{1}[m \in \Theta_{\pi_{ij}}], \quad (7.3)$$

where π_{ij} is the unique shortest path. Phase 2 generalizes this by weighting each route by P_r .

7.3 Forward model (clean turning counts)

For one synthetic OD matrix $\mathbf{T}^{\text{syn}}$ with flattened demand $\mathbf{y}$:

$$\mathbf{x}^{\text{clean}} = \mathbf{A}_{\text{turn}} \mathbf{y}, \quad x_m^{\text{clean}} = \sum_{\ell=0}^{575} A_{\text{turn},m,\ell} y_\ell. \quad (7.4)$$

For a batch of K_{samples} replicates:

$$\mathbf{X}^{\text{clean}} = \mathbf{Y} \mathbf{A}_{\text{turn}}^\top, \quad \mathbf{Y} \in \mathbb{R}^{K_{\text{samples}} \times 576}, \quad \mathbf{X}^{\text{clean}} \in \mathbb{R}^{K_{\text{samples}} \times M}. \quad (7.5)$$

Equation (7.5) is identical in form to Phase 1; only $\mathbf{A}_{\text{turn}}$ changes from binary to fractional. Turning count x_m aggregates OD flows weighted by the probability that each route uses movement m .

7.4 Identifiability and rank analysis

Linear-algebra diagnostics compare Phase 1 and Phase 2 operators:

Table 7.1: Assignment matrix rank comparison (Sioux Falls, $K = 5$, $\theta = 0.1$).

Quantity	$\mathbf{A}_{\text{turn}}^{\text{bin}}$ (Phase 1)	$\mathbf{A}_{\text{turn}}$ (Phase 2)
Shape	(178, 576)	(178, 576)
Nonzeros	1,324	7,326
$\text{rank}(\mathbf{A}_{\text{turn}})$	122	174
Nullity ($576 - \text{rank}$)	454	402
$\kappa(\mathbf{A}_{\text{turn}})$	≈ 14.8	$\approx 1.2 \times 10^{12}$

$$\text{rank}(\mathbf{A}_{\text{turn}}^{\text{bin}}) = 122, \quad \mathcal{N}_{\text{bin}} = 576 - 122 = 454, \quad (7.6)$$

$$\text{rank}(\mathbf{A}_{\text{turn}}) = 174, \quad \mathcal{N} = 576 - 174 = 402. \quad (7.7)$$

Rank increases by 52: probabilistic splitting adds 52 independent column directions in $\mathbf{A}_{\text{turn}}$, improving the information content of turning-count observations for OD recovery. The condition number rises because fractional columns are nearly collinear for some OD pairs—regularization in Part III becomes even more important. Implementation: `src/assignment_rank.py`, `src/rank_analysis.py` singular-value plots in `figures/singular_values.png`.

Implication for Part III. The inverse problem remains underdetermined ($\mathcal{N} = 402 > 0$), but the identifiable subspace is larger. Ridge, Elastic Net, and forward-consistency losses should be evaluated on Phase 2 labels to measure whether the rank gain translates to lower OD recovery error.

Chapter 8

Observation Noise and Dataset Assembly

8.1 Observation noise models

Real loop detectors and camera counts are noisy. Phase 2 supports three observation models applied *after* clean synthesis:

No noise. $\mathbf{x}^{\text{obs}} = \mathbf{x}^{\text{clean}}$.

Gaussian noise.

$$x_m^{\text{obs}} = x_m^{\text{clean}} + \epsilon_m, \quad \epsilon_m \sim \mathcal{N}(0, \sigma^2 (x_m^{\text{clean}})_+), \quad (8.1)$$

with clipping at zero. Parameter σ controls relative noise level.

Poisson noise (default).

$$x_m^{\text{obs}} \sim \text{Poisson}(x_m^{\text{clean}}), \quad (8.2)$$

which respects nonnegativity and heteroscedastic variance $\text{Var}(x_m^{\text{obs}} | \mathbf{x}^{\text{clean}}) = x_m^{\text{clean}}$. Nominal noise levels 0%, 2%, 5%, 10% map to a Poisson scaling parameter when partial noise is desired. Implementation: `src/observation_noise.py`.

Table 8.1: Observation noise models: pros and cons.

Model	Pros	Cons
None	Exact labels for sanity checks	Optimistic ML performance
Gaussian	Simple; additive	Can produce negative counts without clipping
Poisson (chosen)	Standard for count data; mean=variance	Not suitable for overdispersed sensors without extension
Negative binomial	Handles overdispersion	Extra dispersion parameter

Why Poisson. Turning counts are non-negative integer aggregates of trips. Poisson noise is the canonical count-data model and matches the signal-to-noise structure of traffic sensors at moderate volumes (Cascetta). Gaussian noise is retained for ablation.

Network: `sioux-falls.net.xml` $\rightarrow \mathcal{G}$, 178 turns
Route catalog: for each (i, j) : $\mathcal{R}_{ij} \leftarrow K$ shortest simple paths
Logit: $P_r \propto \exp(-\theta c_r)$
Fractional $\mathbf{A}_{\text{turn}}$: $A_{m,\ell} = \sum_r P_r b_{r,m}$
Labels: $\{\mathbf{T}^{\text{syn}}, (k)\}_{k=1}^{K_{\text{samples}}} \xrightarrow{\text{vec}} \mathbf{Y} \xrightarrow{\mathbf{X} = \mathbf{Y} \mathbf{A}_{\text{turn}}^\top} \mathbf{X}^{\text{clean}} \xrightarrow{\text{Poisson}} \mathbf{X}^{\text{obs}}$

Figure 8.1: Phase 2 turning-count dataset pipeline (`generate_turning_counts.py`).

Algorithm 2: Batched turning-count generation (scale $K_{\text{samples}} = 10^6$)

1. Parse network; enumerate 178 turns; build $\mathbf{A}_{\text{turn}}$ (once)
2. Memory-map `synthetic_od.npy` (read-only); never load full $\mathbf{Y}$
3. For each batch b of size B (default $B = 10,000$ – $20,000$):
 - (a) $\mathbf{Y}^{(b)} \leftarrow \text{reshape}(\mathbf{T}^{\text{syn}}[b : b + B])$ as `float32`
 - (b) $\mathbf{X}^{\text{clean},(b)} \leftarrow \mathbf{Y}^{(b)} \mathbf{A}_{\text{turn}}^\top$
 - (c) $\mathbf{X}^{\text{obs},(b)} \leftarrow \text{Poisson}(\mathbf{X}^{\text{clean},(b)})$
 - (d) Write rows to memory-mapped `turning_counts.npy`
4. Streaming mean/std; sampled pairwise diversity; optional figures from aggregates

Figure 8.2: Memory-safe batching (`src/batch_turning.py`). Peak RAM $\approx B(576 + M) \times 4$ bytes ≈ 60 MB at $B = 20,000$.

8.2 End-to-end Phase 2 pipeline

8.3 Scalability: why the system restarted and how it was fixed

The initial Phase 2 implementation loaded all $K_{\text{samples}} = 10^6$ OD matrices into RAM and materialized multiple full copies ($\mathbf{Y}$, $\mathbf{X}^{\text{clean}}$, $\mathbf{X}^{\text{obs}}$), with peak usage exceeding 12–15 GB on a 15 GB machine—triggering the Linux OOM killer.

The production fix (`src/batch_turning.py`) uses:

- **Memory-mapped input:** `np.load(..., mmap_mode='r')` for `synthetic_od_1m.npy` (~ 2.3 GB on disk, not in RAM);
- **Batched matrix multiply:** $\mathbf{X} = \mathbf{Y} \mathbf{A}_{\text{turn}}^\top$ in chunks of size B ;
- **Memory-mapped output:** incremental write to `turning_counts.npy` (~ 680 MB);
- **Streaming statistics:** online mean/variance; sampled pairwise distances;
- **Auto-batching:** enabled for $K_{\text{samples}} \geq 50,000$; `--batch-size` configurable.

Benchmark: $K_{\text{samples}} = 10^6$, $B = 20,000$, $K = 5$, $\theta = 0.1$ completes in ≈ 16 s with peak batch RAM ≈ 60 MB.

8.4 Outputs and deliverables

8.5 Validation

Forward consistency. For each batch, verify $\|\mathbf{x}^{\text{clean}} - \mathbf{A}_{\text{turn}} \mathbf{y}\|_\infty < 10^{-3}$ on a probe sample.

Structural checks. 178 turns; 576 OD columns; fractional entries in $[0, 1]$; shapes $(K_{\text{samples}}, 178)$ and $(K_{\text{samples}}, 576)$ for targets loaded separately from Part I.

Table 8.2: Part II deliverables (outputs/turning_counts_1m/).

File	Content
A_turn.npy	Fractional assignment matrix (178, 576), float32
turning_counts.npy	$\mathbf{X}^{\text{clean}}$, shape (K_{samples} , 178)
turning_counts_noisy.npy	$\mathbf{X}^{\text{obs}}$ (Poisson default)
route_catalog.pkl	K -path catalog + logit probabilities
turning_movements.csv	Movement index (e^{in} , e^{out} , m)
assignment_analysis.json	Rank comparison, validation, timings
figures/	Histograms, heatmap, singular values, rank bar chart
turning_dataset_summary.md	Human-readable report

8.6 Sensitivity analysis

Grid over (K, θ) measures $\text{rank}(\mathbf{A}_{\text{turn}})$, nullity, $\kappa(\mathbf{A}_{\text{turn}})$, and maximum column entry. Recommended production defaults from Sioux Falls runs: $K = 5$, $\theta = 0.1$. Increasing K at fixed θ adds routes but with diminishing rank gains beyond $K = 5$; increasing θ moves Phase 2 toward Phase 1. Enable via `--run-sensitivity` (optional; re-enumerates paths per K).

8.7 Part II evaluation highlights

On $K_{\text{samples}} = 10^6$ synthetics (Dirichlet+IPF Part I, Phase 2 Part II):

- $\text{rank}(\mathbf{A}_{\text{turn}}) = 174$ vs. Phase 1 rank 122 (+52);
- mean turning count per movement $\approx 3,051$ (batch mean over first 1000 samples);
- OD pairwise diversity (sampled) $\bar{d}_{\text{OD}} \approx 545$;
- turning pairwise diversity $\bar{d}_{\text{turn}} \approx 894$;
- mean inter-turn correlation (50-feature sample) ≈ 0.01 .

Turning counts remain a linear compression of OD demand, but probabilistic assignment *increases* turning-space separation relative to Phase 1 while preserving the exact forward map needed for supervised training.

Chapter 9

Implementation Reference (Part II)

Table 9.1: Part II: key modules and scripts.

Module / script	Role
<code>generate_turning_counts.py</code>	CLI orchestrator (Phase 2)
<code>src/network_parser.py</code>	SUMO $\rightarrow$ NetworkX graph
<code>src/turning_movements.py</code>	178 movements + CSV export
<code>src/k_shortest_paths.py</code>	K shortest simple paths per OD
<code>src/route_choice.py</code>	Multinomial logit probabilities
<code>src/fractional_assignment.py</code>	Route turn vectors + fractional $\mathbf{A}_{\text{turn}}$
<code>src/assignment_rank.py</code>	Binary vs. probabilistic rank analysis
<code>src/observation_noise.py</code>	None / Gaussian / Poisson
<code>src/batch_turning.py</code>	Memory-mapped batched generation
<code>src/turning_statistics.py</code>	Stats, figures, sensitivity grid
<code>generate_dataset.py</code>	Phase 1 baseline (binary $\mathbf{A}_{\text{turn}}$)
<code>src/route_assignment.py</code>	Shortest-path routes
<code>src/build_assignment_matrix.py</code>	Binary $\mathbf{A}_{\text{turn}}^{\text{bin}}$

Example command (1M samples).

```
python generate_turning_counts.py \  
  --network sioux-falls.net.xml \  
  --od outputs/od_generator/synthetic_od_1m.npy \  
  --output-dir outputs/turning_counts_1m \  
  --k_paths 5 --theta 0.1 --noise poisson \  
  --batch-size 20000 --skip-plots
```

Legacy Phase 1 command (binary baseline).

```
python run_dataset_pipeline.py \  
  --od synthetic_od_100000.npy --output-dir outputs/
```


Part III

Machine Learning Model Selection and Training

Chapter 10

Inverse Problem Formulation

10.1 Problem statement

Given turning-count observation $\mathbf{x}^{\text{obs}} \in \mathbb{R}^{178}$, estimate OD vector $\mathbf{y} \in \mathbb{R}_{\geq 0}^{576}$ such that

$$\mathbf{x}^{\text{obs}} \approx \mathbf{A}_{\text{turn}} \mathbf{y}, \quad (10.1)$$

where $\mathbf{A}_{\text{turn}}$ is the *fractional* Phase 2 assignment matrix (rank = 174). Because $\mathbf{A}_{\text{turn}}$ is rank-deficient ($\mathcal{N} = 402$), the problem is *underdetermined*. Training uses synthetic pairs $(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})$ where $\mathbf{x}^{\text{clean},(k)} = \mathbf{A}_{\text{turn}} \mathbf{y}^{(k)}$ exactly and $\mathbf{x}^{\text{obs},(k)}$ may include Poisson noise.

10.2 Learning paradigms considered

Table 10.1: ML paradigms for OD estimation from turning counts.

Paradigm	Pros	Cons
Linear regression + regularization	Fast; interpretable; encodes $\mathbf{A}_{\text{turn}}$ structure via priors	Limited nonlinear capacity
Physics-informed (constrained LS)	Enforces $\mathbf{x} = \mathbf{A}_{\text{turn}} \mathbf{y}$; minimal black-box	Needs strong priors; null-space ambiguity
Deep neural network	Flexible; scales to large K	Risk of overfitting; may violate nonnegativity/sparsity
Gradient boosting	Strong tabular performance; handles interactions	Less interpretable; slower at inference
Bayesian inversion	Uncertainty quantification	Computationally heavier

Chapter 11

Candidate Models and Selection Rationale

11.1 Model zoo

Table 11.1 lists candidate models evaluated for this project. The **recommended starting stack** is Ridge regression (baseline) followed by a regularized MLP if nonlinear effects are needed.

Table 11.1: Candidate ML models: pros, cons, and suitability for turning-count $\rightarrow$ OD inversion.

Model		Pros	Cons	Selected?
Ordinary squares	least	Closed form; baseline	Ill-posed ($M < N^2$); no priors	No
Ridge regression		Stable under collinearity; analytic solution; fast	Linear only	Yes (baseline)
Lasso / Elastic Net		Promotes sparsity in $\mathbf{y}$; matches OD structure	Linear; tuning λ	Yes (sparse OD)
Non-negative with $\ \mathbf{y}\ _2, \ \mathbf{y}\ _1$	LS	Enforces $y_\ell \geq 0$	Still linear; null-space	Yes (extension)
Tikhonov on $\mathbf{A}_{\text{turn}}$		Explicit use of forward operator	Requires choosing penalty on null-space	Yes (physics baseline)
MLP (feedforward NN)		Nonlinear; scales to large K	Needs NN constraints for ≥ 0 ; tuning	Yes (nonlinear)
Random Forest / XGBoost		Strong on tabular data; robust	178-dim input ok but 576-dim output needs multi-output; slow	Secondary
CNN on 24×24		Exploits grid structure if reshaped	Turning counts lose spatial grid; awkward	No
GNN on network		Natural for transport topology	Heavy implementation; Part IV	Future

11.2 Why Ridge + Elastic Net as primary baselines

1. **Ill-posedness:** with nullity $\mathcal{N} = 402$ under Phase 2 (454 under Phase 1), unregularized inversion is unstable. Ridge (L_2) shrinks coefficients in the identifiable subspace.
2. **Sparsity:** Elastic Net adds L_1 penalty, encouraging zero flows on off-support cells—consistent with $\mathbf{T}^{\text{base}}$ structure.

3. **Speed:** analytic / convex optimization trains in seconds even for $K = 10^6$.
4. **Interpretability:** coefficients relate directly to turning-movement sensitivities.

11.3 Why MLP as nonlinear extension

Turning counts are *linear* in OD under both Phase 1 and Phase 2 forward models, so a linear model should perform well on synthetics generated with the same $\mathbf{A}_{\text{turn}}$. An MLP is retained because:

- real data may include measurement noise and model mismatch;
- nonlinear calibration (e.g. log-transform of counts) may help;
- it stress-tests whether the synthetic dataset spans enough variation for nonlinear methods to add value over Ridge.

Constraints: ReLU + nonnegative output layer (or softplus); optional sparsity penalty on outputs.

11.4 Why not simulation-based or equilibrium models for training labels?

Traffic simulators produce realistic flows but are slow, stochastic, and couple OD with route choice nonlinearly. For generating 10^6 labeled pairs, the linear $\mathbf{A}_{\text{turn}}$ model is *sufficient and necessary* for throughput and for maintaining a known ground-truth Jacobian structure.

Chapter 12

Training Protocol and Evaluation

12.1 Recommended data split

- Generate $K_{\text{samples}} = 10^6$ synthetics (Part I) and corresponding $(\mathbf{X}^{\text{obs}}, \mathbf{Y})$ (Part II, Phase 2);
- Split 80/10/10 train/validation/test by sample index (seed-controlled);
- **Never** mix base OD as a test sample—it is not in the synthetic distribution.

12.2 Feature and target preprocessing

- Inputs $\mathbf{X}$: standardize turning counts (178 features) using training-set mean/std;
- Targets $\mathbf{Y}$: optional $\log(1 + y)$ transform; clip negatives to zero after prediction;
- Post-process: project predictions onto sparsity mask $\mathbf{M}$ and zero diagonal.

12.3 Loss functions

Table 12.1: Loss functions for OD recovery.

Loss	Formula (single sample)	Use case
MSE	$\ \hat{\mathbf{y}} - \mathbf{y}\ _2^2$	Default
MAE	$\ \hat{\mathbf{y}} - \mathbf{y}\ _1$	Robust to outliers
Forward-consistency	$\ \mathbf{x} - \mathbf{A}_{\text{turn}}\hat{\mathbf{y}}\ _2^2$	Physics-informed fine-tuning
Combined	$\text{MSE} + \lambda\ \mathbf{x} - \mathbf{A}_{\text{turn}}\hat{\mathbf{y}}\ _2^2$	Enforce observations

For ill-posed inversion, **combined loss** (supervised MSE + forward-consistency) is recommended when deploying beyond pure synthetic evaluation.

12.4 ML evaluation metrics

On held-out test set:

$$\text{RMSE}_{\text{OD}} = \sqrt{\frac{1}{N^2} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2}, \quad (12.1)$$

$$\rho_{\text{OD}} = \text{corr}(\mathbf{y}, \hat{\mathbf{y}}), \quad (12.2)$$

$$\text{RMSE}_{\text{turn}} = \|\mathbf{x} - \mathbf{A}_{\text{turn}}\hat{\mathbf{y}}\|_2, \quad (12.3)$$

$$\text{MAE}_{\text{marginal}} = \text{production/attraction error vs. ground truth.} \quad (12.4)$$

12.5 Hyperparameter tuning (Part III)

- Ridge / Elastic Net: cross-validate λ on log-spaced grid;
- MLP: hidden layers $\{128, 256\}$, dropout 0.1–0.3, Adam, early stopping on validation MSE;
- Compare against **Tikhonov baseline**: $\hat{\mathbf{y}} = (\mathbf{A}_{\text{turn}}^\top \mathbf{A}_{\text{turn}} + \lambda I)^{-1} \mathbf{A}_{\text{turn}}^\top \mathbf{x}$.

12.6 Planned implementation (Part III)

The following modules are reserved for the next development stage:

```
src/ml/  
  datasets.py      # load X, Y; train/val/test split  
  baselines.py     # Ridge, Elastic Net, Tikhonov  
  mlp_model.py     # PyTorch or sklearn MLP  
  train.py         # training loop + logging  
  evaluate.py      # RMSE, correlation, forward error
```

Part IV

Future Work and Extensions

Chapter 13

Roadmap for Subsequent Chapters

This part is intentionally brief. As the project progresses, the following chapters will be added:

Table 13.1: Planned future chapters (Part IV+).

Chapter (planned)	Topic
Real-data validation	Compare predictions against held-out counts / partial OD
Overdispersed noise	Negative binomial / compound Poisson on $\mathbf{x}^{\text{obs}}$
User equilibrium extension	Nonlinear forward model; simulation labels
Uncertainty quantification	Bayesian or ensemble CIs on $\mathbf{y}$
Deployment	Inference pipeline from field turning counts

Completed since initial roadmap. Multi-path fractional assignment (K -shortest + logit) and Poisson observation noise are now implemented in Part II (`generate_turning_counts.py`).

Remaining limitations.

- Single observed $\mathbf{T}^{\text{base}}$; no temporal dynamics;
- No spatial correlation in Dirichlet shocks;
- Logit route choice without congestion feedback (static costs);
- Nullity $\mathcal{N} = 402$ still limits unique OD recovery without priors;
- Part III training code not yet implemented (documented design only).

Conclusion

Part I produces diverse, structurally valid synthetic OD matrices via Dirichlet sampling, marginal perturbation, and IPF—scalable to 10^6 samples. Part II constructs turning-count observations through a route-choice-aware fractional operator $\mathbf{A}_{\text{turn}}$ (rank = 174, up from 122 under binary shortest-path assignment) on the Sioux Falls network, with optional Poisson noise and memory-mapped batch generation. Part III specifies Ridge/Elastic Net baselines and MLP extensions for the underdetermined inverse problem, with regularization and forward-consistency losses motivated by null-space dimension $\mathcal{N} = 402$. Future parts will add real-data validation, overdispersed noise, and equilibrium-based assignment models.

Bibliography

- [1] R. Furness, *Trip forecasting*, Traffic Engineering and Control, 1962.
- [2] W. E. Deming and F. F. Stephan, *On a least squares adjustment of a sampled frequency table when the expected marginal totals are known*, Annals of Mathematical Statistics, 1940.
- [3] J. de D. Ortúzar and L. G. Willumsen, *Modelling Transport*, 4th ed., Wiley, 2011.
- [4] E. Cascetta, *Estimation of trip matrices from traffic counts and survey data*, Transportation Research Part B, 1984.